

# 0.76.6. Un déclencheur en (0, 0) : le D file à droite

Le 0.76.5, plus un D : il apparaît au bord gauche, sur la ligne 14, et file vers la droite ; au bord, il repart de gauche.

## 1. Ce qu'il faut comprendre

C'est le 0.76.5, plus un D (ALPHABET[3]).

**Ce qui est nouveau ici : le mouvement inverse du C.** Le D avance d'une colonne (dx++) à chaque tic, et arrivé à la colonne 19, il **repart de la colonne 0** : `if (dx == 19) { dx = 0; } else { dx++; }`.

**Tout part du même déclencheur** : en (0, 0), le A fait apparaître quatre lettres, et chacune a son mouvement. Le A, lui, suit toujours la croix.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2871 octets** de cartouche.

### 1. UNE VARIABLE — X

```
uint8_t x = 10;    // le A part de (10, 0)
```

### 2. UNE VARIABLE — Y

```
uint8_t y = 0;
```

### 3. UNE VARIABLE — ACTIF

```
uint8_t actif = 0;    // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

### 4. UNE VARIABLE — BX

```
uint8_t bx = 10;    // le B : sa colonne...
```

### 5. UNE VARIABLE — BY

```
uint8_t by = 8;    // ...et sa ligne. Il apparaît au milieu, (10, 8).
```

### 6. UNE VARIABLE — BPAS

```
uint8_t bpas = 0;    // où il en est dans son tour : du pas 0 au pas 15
```

### 7. UNE VARIABLE — BIMAGES

```
uint8_t bimages = 0;    // le chronomètre du B, en images
```

### 8. UNE VARIABLE — BX2

```
uint8_t bx2 = 14;    // le second B : il part du coin opposé, (14, 12)...
```

#### 9. UNE VARIABLE — BY2

```
uint8_t by2 = 12;
```

#### 10. UNE VARIABLE — BPAS2

```
uint8_t bpas2 = 8;    // ...donc déjà à mi-tour : il commence par aller à gauche
```

#### 11. UNE VARIABLE — CX

```
uint8_t cx = 19;      // le C : il apparaît en (19, 3), au bord droit
```

#### 12. UNE VARIABLE — CY

```
uint8_t cy = 3;
```

#### 13. UNE VARIABLE — DX

```
uint8_t dx = 0;       // le D : il apparaît en (0, 14), au bord gauche
```

#### 14. UNE VARIABLE — DY

```
uint8_t dy = 14;
```

## 15. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]);          // le B apparaît au milieu
            poser(14, 12, ALPHABET[1]);        // le second B, au coin opposé
            poser(19, 3, ALPHABET[2]);         // le C apparaît au bord droit
            poser(0, 14, ALPHABET[3]);         // le D apparaît au bord gauche
        }

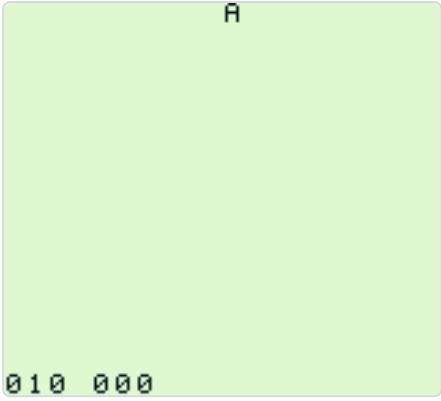
        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++;                          // le chronomètre avance
            if (bimages == 5) {                 // toutes les 5 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
                if (bpas2 < 4) { un_pas(bx2, by2, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas2 < 8) { un_pas(bx2, by2, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas2 < 12) { un_pas(bx2, by2, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx2, by2, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas2++;
                if (bpas2 == 16) { bpas2 = 0; } // le tour est fini : on recommence
                effacer(cx, cy, 1);             // le C : un pas à GAUCHE...
                if (cx == 0) { cx = 19; } else { cx--; } // ...et au bord gauche, il repart de droite
                poser(cx, cy, ALPHABET[2]);
                effacer(dx, dy, 1);             // le D : un pas à DROITE...
                if (dx == 19) { dx = 0; } else { dx++; } // ...et au bord droit, il repart de gauche
                poser(dx, dy, ALPHABET[3]);
            }
        }
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

En (0, 0), deux B tournent, un C file à gauche en haut, un D file à droite en bas.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le D est apparu
- ✓ il va vers la DROITE (4 → 8)
- ✓ et le C va toujours à gauche

Ensuite : **0.76.7. Un déclencheur en (0, 0) : actif — en simple** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++